

Enhancing Efficiency in Text Splitting: Exploring Semantic Clustering Methods.

7 min read · Mar 1, 2024



Frits Traets

Follow



Listen



Share

To harness unstructured data, such as text, vector databases are frequently employed. This process involves initially splitting larger documents into meaningful text chunks, which are then used to generate embeddings. This article explores a machine learning approach to segment text based on semantic information. We observe that our model is able to accurately split long pieces of text into semantically homogenous paragraphs.

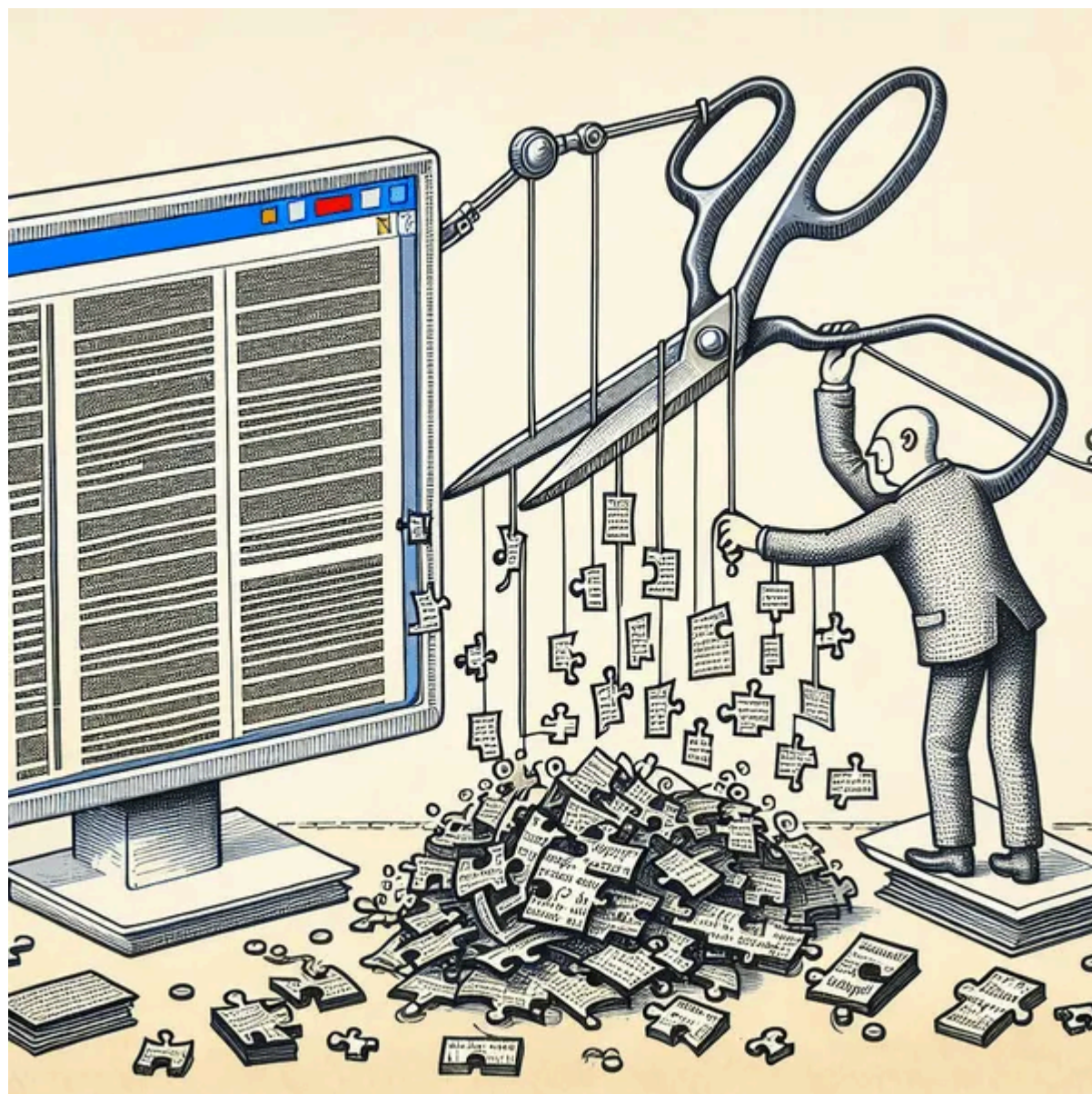


Fig 1. Splitting text (dall-e-3)

Introduction

Text splitting involves the conversion of unstructured text data into meaningful units. In the realm of Natural Language Processing (NLP), text splitters play a crucial role in pre-processing textual data for tasks such as machine translation, text summarization, or named entity recognition among others.

In the context of RAG (Retrieval-Augmented Generative) applications, adopting a pragmatic text splitting strategy is also a key consideration. Efficiently dividing input text into manageable “chunks”, for which embeddings can be computed, is important in order to ensure relevant information retrieval.

At a high level, methods to achieve this typically fall into one of two categories. The first, rule-based methods, make use of explicit separators such as punctuation, space characters, font and text formatting, or regular expressions to partition text

into chunks. The second category, semantic clustering methods, leverages the inherent meaning embedded in the text to guide the chunking process. To do so, machine learning algorithms can be used to discern context and infer natural divisions within the text. In this document we explore the latter strategy.

Goals

In a typical RAG application, a vector database is used to retrieve relevant information, which is then augmented to the user input such that a Large Language Model (LLM) can generate contextually relevant output. In theory, it is possible to retrieve an entire corpus as long as it fits within the context window of the LLM. However, considering the attention span of an LLM, it is also beneficial to minimize the retrieval of irrelevant information. Furthermore, the act of splitting a text can disrupt a narrative, leading to the potential loss of contextual information or, in the worst case, a shift in the conditional meaning of specific textual segments.

Get Frits Traets's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

☐ Remember me for faster sign in

To achieve effective information retrieval, it is thus advisable to find a balance between relevance and completeness. Ideally, when preparing chunks for a vector database in RAG applications, a text-splitting process should:

1. Result in text chunks that exhibit semantic homogeneity, ensuring consistent and focused meaning within each chunk.
2. Maximize the semantic differences between text chunks.
3. Minimize the loss of contextual information during the splitting process.

General strategy

As it would be hard to optimise a splitting process monitoring the previously described characteristics, another way is to start from a dataset of text chunks that

already exhibit these desired characteristics. We employed a freely available [Kaggle dataset](#) with over 38,000 abstracts of scientific papers for training data. Each abstract contains sentences related to the same topic, and as such share a semantic unity. By labeling the transitions between different abstracts as split points, we aimed to construct a labeled dataset that enables a model to recognize a splitting pattern.

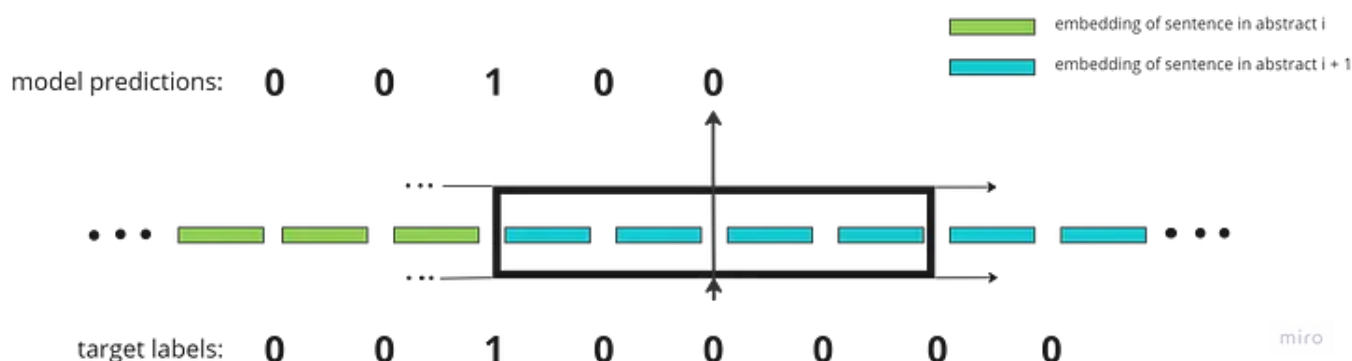


Fig 2. Sliding window approach.

Consequently, developing a semantic splitter model becomes training a classification model to predict whether or not to split given some features.

Feature Engineering

As features, we utilized numerical representations of sentences obtained by calculating embeddings through the '[text-embedding-ada-002](#)' model from OpenAI. These embeddings, with a dimensionality of 1536, depict the sentences' contextual information. We treat the data as a sequence by concatenating all sentences (or their respective embeddings). Subsequently, we employ a sliding window approach, selecting a specific number of consecutive sentences, to construct our dataset as can be seen in Fig 2. With for example a sliding window of $N_{\text{window}} = 4$ sentences, the following dataframe could be produced:

split 1	split 2	split 3	Y_split1	Y_split2	Y_split3
			0	0	1
			0	1	0
			1	0	0
			0	0	0
⋮					
⋮					

$N_{\text{features}} = N_{\text{window}} * 1536$

miro

Fig 3. Concatenated embeddings and split points.

Note that there are $N_window - 1$ possible splits, making it a multilabel classification task. However we've opted focus on a singular split (Y_split2) — as our target label. As such, we have a number of embeddings ($N_window / 2$) before and after the possible split. Instead of concatenating these embeddings, we reasoned that computing a single embedding for multiple sentences would be more efficient. In the case of $N_window = 4$, as illustrated in Fig 3, we calculate one embedding for the two sentences preceding the potential split point and another for the two sentences following it. This approach reduces the dimension of the features to $2 * 1536$, regardless of the number of sentences in the window (N_window), as depicted in Fig 4.

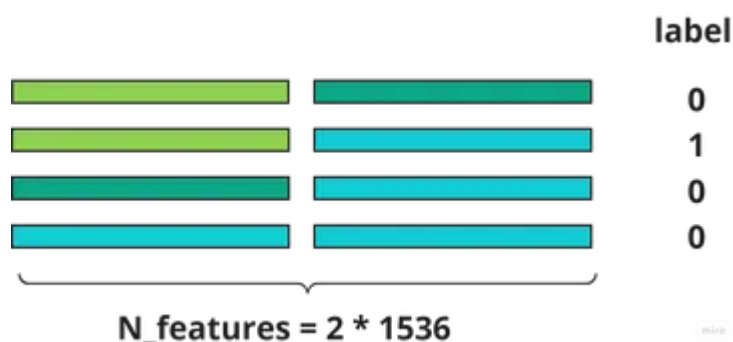


Fig 4. Multiple sentences as embedding.

We can even go a step further. Since a split indicates a semantic difference between two chunks of text, we can calculate the difference vector for each row to serve as input for a model. These vectors are derived by subtracting the embedding of sentences following the potential split from those preceding it. As such, the number of features is reduced to the dimension of one embedding.

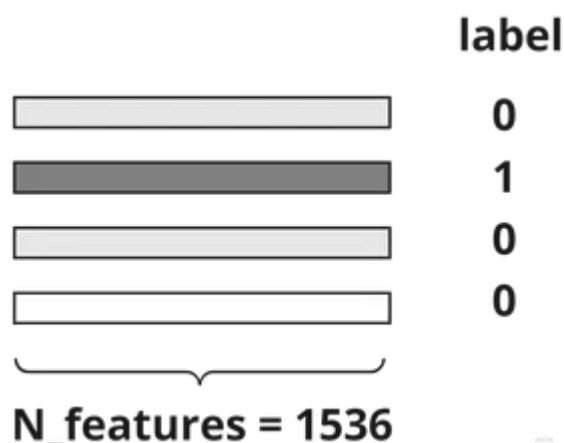


Fig 5. Difference vectors as features.

We anticipate that when two pieces of text are naturally separated (labeled as 1), the resulting difference vector would, on average, exhibit a larger magnitude compared to difference vectors stemming from within the same segment. Our observation confirms this expectation, as shown below.

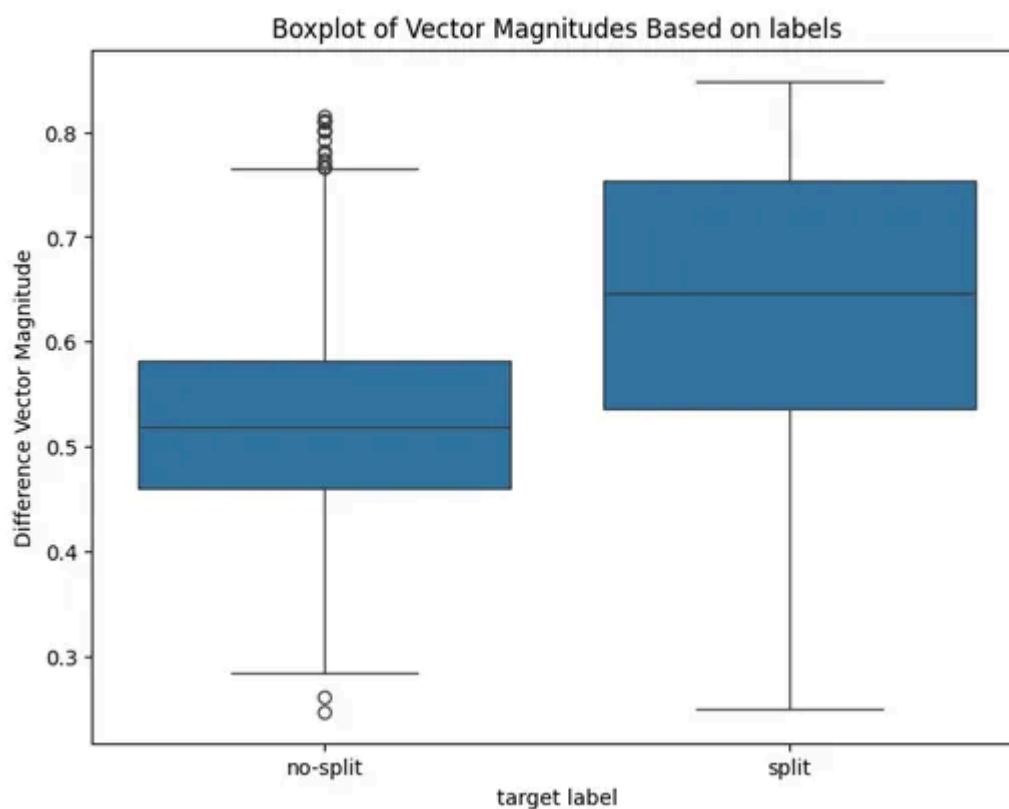


Fig 6. Boxplot vector magnitudes per label.

Dimensionality Reduction

As a concluding step in feature manipulation, we applied Principal Component Analysis (PCA) to further reduce the dimensionality of the features, ensuring that 95% of the cumulative variance is retained. This resulted in a reduction of feature dimensionality from 1536 to 693.

Split data

Lastly, a training and test set is created to train and evaluate the model predictions. By default 80% of the observations will be in the training set and 20% will be in the test set. Hereby the observations were also filtered to achieve a balanced dataset with an equal count of labels 0 and 1.

Modelling

Three types of models were tried as classifiers: Random Forest, Gradient Boosting, and Support Vector Machines (SVM). To select the hyperparameters a grid search was performed on a subset of the data (N = 1000). The models were then fit with the

best hyperparameters on the training data containing (N = 8000) balanced samples. Afterwards, the performance of each model was tested on the test data (N = 2000), of which the results can be seen below:

	Random Forest	Gradient Boosting	Support Vector Machines
Precision	0.97	0.98	0.93
Recall	0.98	0.99	0.96
F1 Score	0.97	0.99	0.94

Fig 8. Performance on **training** set.

Open in app ↗

Sign up

Sign in

Medium

Search



Recall	0.85	0.88	0.93
F1 Score	0.85	0.87	0.92

Fig 9. Performance on **test** set.

The random forest and gradient boosting algorithms exhibited a slightly better fit to the training data compared to the support vector machine (SVM). However, on the test data, the SVM outperformed the other algorithms, suggesting a potential overfitting issue with the random forest and gradient boosting models. Given that the SVM was the fastest to estimate and produced more than satisfactory results, we opted for it as the chosen model to proceed with.

Validating the model

When testing the model on various unseen text sources, we observed that while the algorithm excelled with data resembling the training set (i.e. scientific abstracts), its performance was less robust when faced with non-academic topics. In response, we retrained the model using a sample from a dataset containing paragraphs from Wikipedia. This adjustment significantly enhanced the model's ability to generalize across various contents. An illustrative example of its improved performance is presented below.

Example

We inputted a continuous text encompassing three topics (blue, green, orange) into the model without any explicit separators. The model successfully segmented the

three distinct topics (and for some reason, it also isolated a single sentence from the blue text). The model's split predictions are indicated with ***SPLIT***.

Semantic text segmentation using a machine learning approach

To harness unstructured data, such as text, vector databases are often employed. This process requires first splitting larger documents into meaningful text chunks, which can then be used to generate embeddings. This article explores a machine learning approach using embeddings to segment text based on semantic information.

SPLIT

We conclude that this method yields promising results.

SPLIT

Barack Hussein Obama II (/bəˈrɑːk huːˈseɪn ʊˈbɑːmə/ ⓘ bə-RAHK hoo-SAYN oh-BAH-mə;[1] born August 4, 1961) is an American politician who served as the 44th president of the United States from 2009 to 2017. A member of the Democratic Party, he was the first African-American president in U.S. history. Obama previously served as a U.S. senator representing Illinois from 2005 to 2008, as an Illinois state senator from 1997 to 2004, and as a civil rights lawyer and university lecturer.

SPLIT



Artificial intelligence (AI) is the intelligence of machines or software, as opposed to the intelligence of other living beings, primarily of humans. It is a field of study in computer science that develops and studies intelligent machines. Such machines may be called AIs.

Fig 10. Model splitting example.

Overall, when assessing the model's predictions on unseen text, we are pleased with the outcomes. When using academic abstracts as data, the model demonstrated its ability to effectively split text discussing closely related topics. In the case of

Wikipedia abstracts, the model could accurately divide text based on the subject matter. Consequently, we are optimistic that this approach can be applied to various specific splitting problems, given that there is access to appropriate example data.

Conclusion

Utilizing embeddings as features in traditional machine learning algorithms appears to be promising approach to split text based on semantic information. Overall, the SVM algorithm delivered excellent predictions, particularly when the content resembled the training data. This limitation can however be addressed by incorporating richer and more diverse training data that closely aligns with the properties of the desired splitting characteristics.

[Semantic Segmentation](#)[NLP](#)[Text Splitter](#)[Retrieval Augmented](#)[Naturallanguageprocessing](#)[Follow](#)

Written by Frits Traets

2 followers · 1 following

Responses (1)



Write a response

What are your thoughts?



Yy

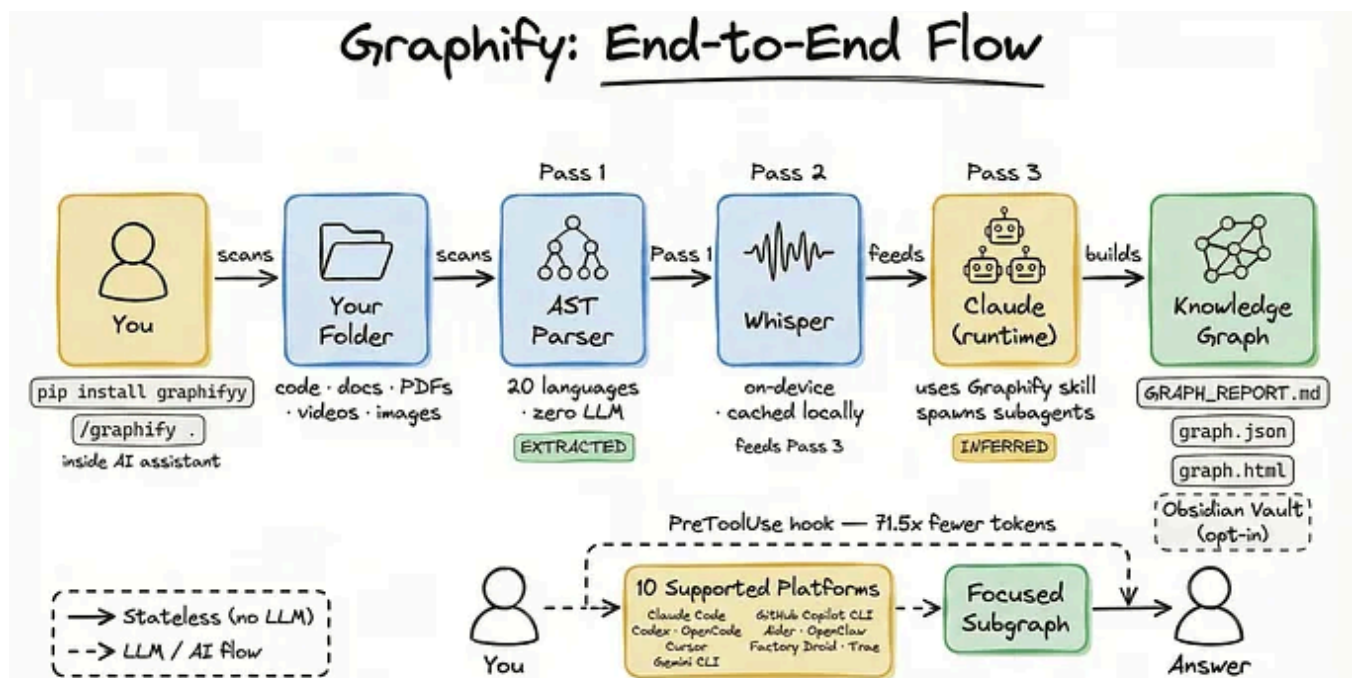
Jun 19, 2024 (edited)



Hi, interesting read! I am currently exploring about better chunking methods and chanced uppn your post on the use of machine learning for chunking text. Are you able to provide the source code that you have used, I would like to take a look!

[Reply](#)

Recommended from Medium



In Agentic Builders by Ana Bildea, PhD

How to Use Graphify: Turn Any Folder Into a Knowledge Graph

A step-by-step guide to using Graphify, the open-source tool that builds a queryable knowledge graph



Apr 11



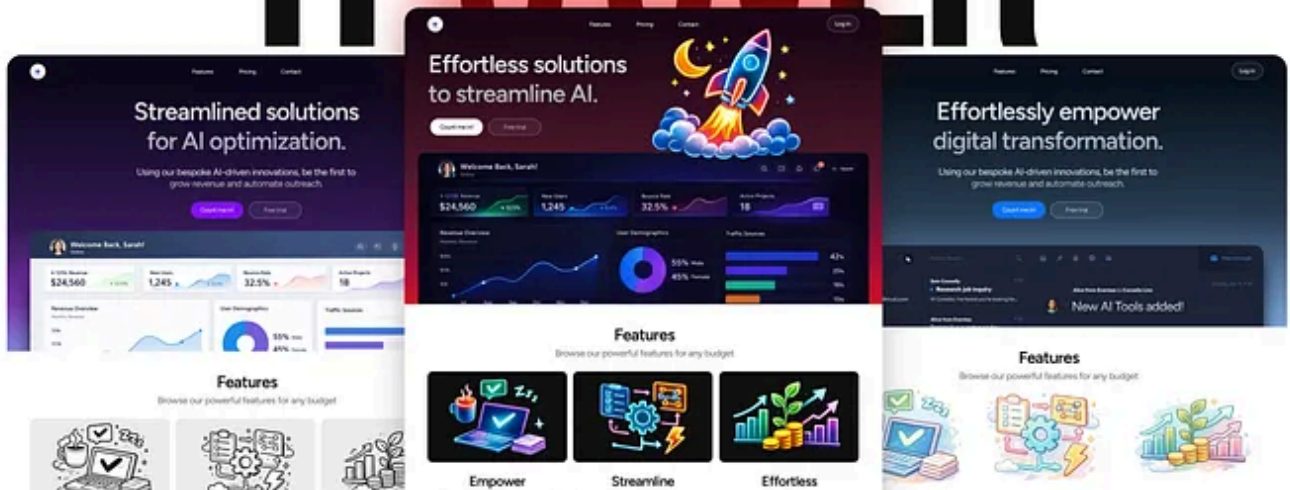
2K



11



IT'S OVER



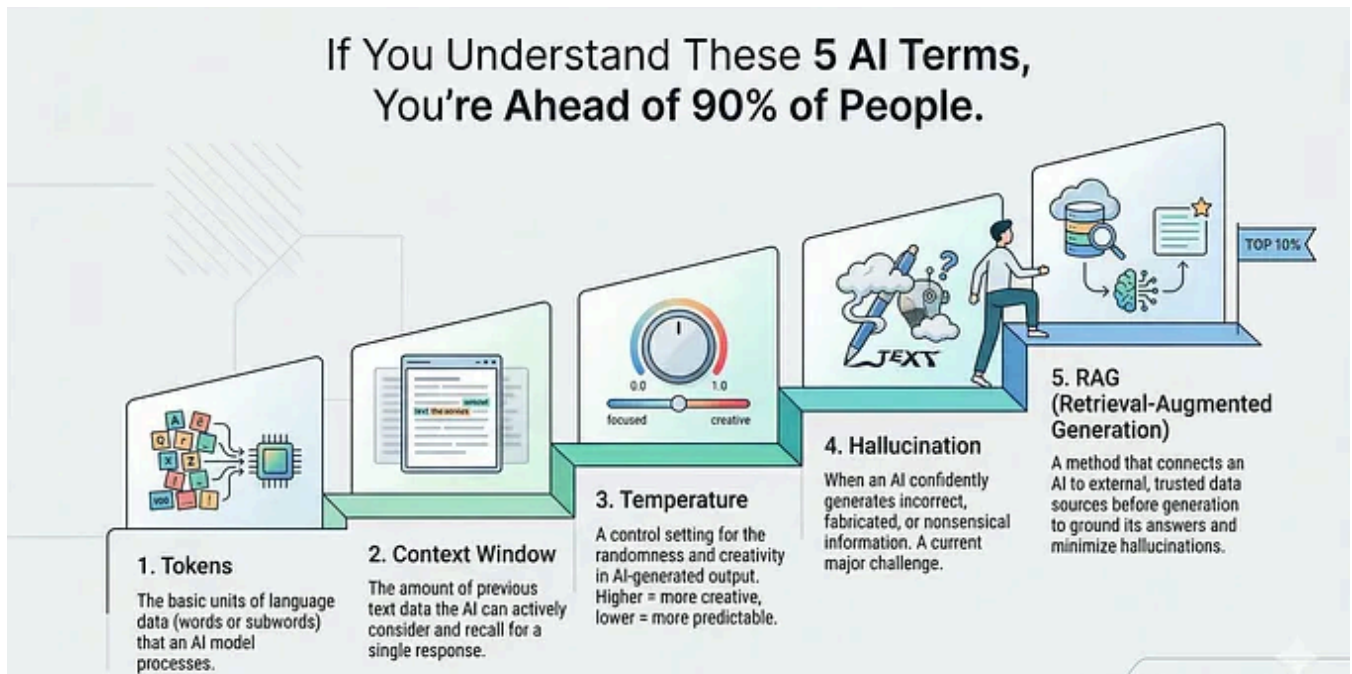
 Michal Malewicz 

Vibe Coding is OVER.

Here's What Comes Next.

★ Mar 24 🤝 7.9K 💬 303





 In Towards AI by Shreyas Naphad

If You Understand These 5 AI Terms, You're Ahead of 90% of People

Master the core ideas behind AI without getting lost

★ Mar 29 🖱 15.4K 💬 310



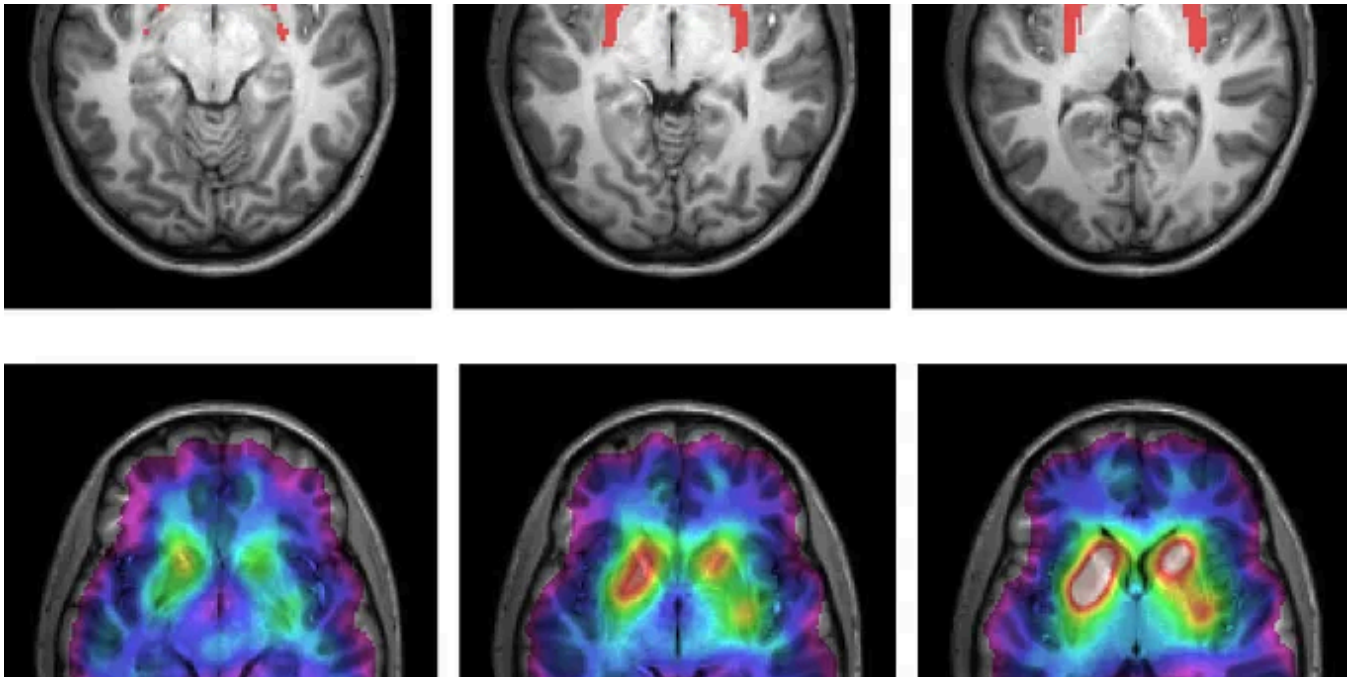
 In UX Planet by Nick Babich 

MCP is Dead

Why you should avoid using MCP in Claude Code and what to use instead

★ Apr 6 🖱 1.5K 💬 94



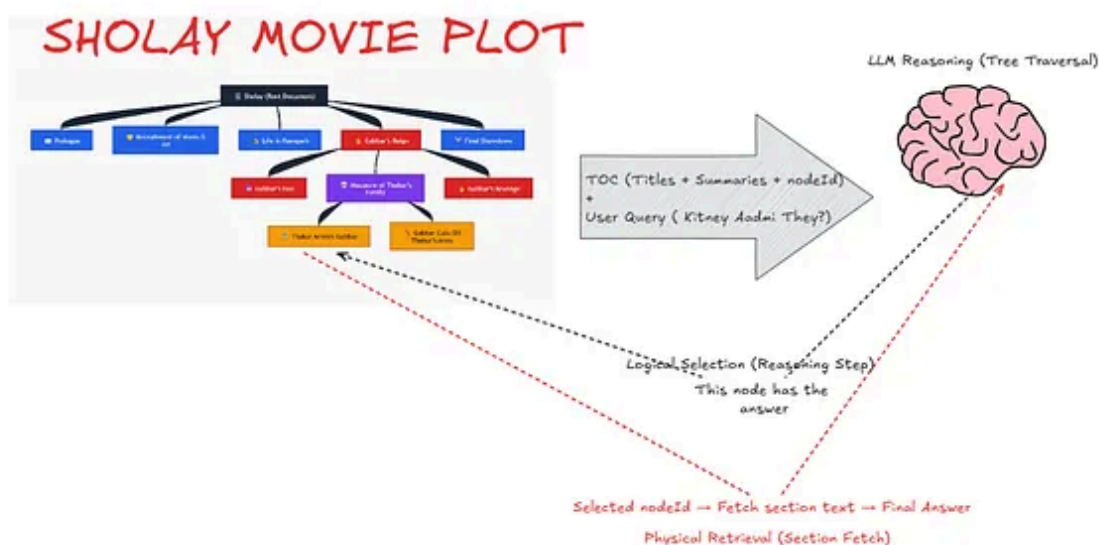


In Write A Catalyst by Dr. Patricia Schmidt 🧠

As a Neuroscientist, I Quit These 5 Morning Habits That Destroy Your Brain

Most people do #1 within 10 minutes of waking (and it sabotages your entire day)

🌟 Jan 14 🖱 49K 💬 1015



Vishal Mysore

What Is PageIndex? How to Build a Vectorless RAG System (No Embeddings, No Vector DB)

PageIndex is a vectorless, reasoning-based Retrieval-Augmented Generation (RAG) approach that retrieves answers from long documents without...

Mar 1  307  12



See more recommendations